NexoLog S.A.S. — Guía de Arquitectura y Decisiones de Diseño

Manual técnico integral: del análisis de insumos al modelo de objetos en memoria

Docente: Ing. Andrés Eduardo Paternina Ariza Esp. **Estado de Solución:** Compilación con 0 errores y ejecución limpia

1. El Problema Real y los 5 Archivos Fuente

NexoLog S.A.S. opera como **plataforma de intermediación logística** entre comercios remitentes (Comercio Caribe, Comercio Andino, Comercio Sierra) y las empresas transportadoras encargadas de la entrega. El núcleo del problema técnico radica en recibir **cinco archivos heterogéneos con datos inconsistentes** y consolidarlos en un modelo de objetos coherente sin abortar la ejecución ante errores.

Archivo Fuente	Formato	Rol y Contenido en el Sistema
pedidos_caribe.json	JSON	Solicitudes de Comercio Caribe (COM-01) y Comercio Sierra (COM-03). Contiene lotes de guías con pesos, distancias y destinatarios.
pedidos_andino.xml	XML	Solicitudes de Comercio Andino (COM-02). Mismo contrato funcional que el anterior, pero estructurado bajo etiquetas XML.
acuerdos.json	JSON	Condiciones comerciales: Tarifa por Peso ($3.500/kg), Tarifa por Distancia ($120/km) o Tarifa Fija ($25.000).
seguimiento_norte.json	JSON	Acontecimientos logísticos de Transportadora Norte (REC, RUT, ENT, DEV).
seguimiento_andina.xml	XML	Acontecimientos logísticos de Transportadora Andina serializados en XML.

■■ Las 4 Inconsistencias Deliberadas del Sistema

1. **Desorden Cronológico:** Hitos de entrega que aparecen físicamente antes de la salida a ruta. Se resuelve ordenando la colección de acontecimientos por FechaHora ascendente antes de procesar.
2. **Datos Inválidos de Frontera:** Paquetes con peso ≤ 0 kg o valores declarados negativos. El sistema captura la excepción de negocio (NexoLogException), crea un RegistroInconsistencia y continúa con los registros sanos.
3. **Duplicados vs Conflictos:** Guías con datos idénticos se omiten sin alterar los cálculos; guías con el mismo número pero distinta información generan un conflicto de identificación en auditoría.
4. **Liquidación Condicionada:** Únicamente los despachos con estado final **Entregado** son facturables. Los devueltos y pendientes generan cobro de $0.

2. Las 5 Decisiones de Diseño Arquitectónico

A. ¿Por qué NO usamos 3 factorías vacías (Modalidad, Tarifa, Acontecimiento)?

El enunciado solicita la aplicación del **Patrón Factory** (en singular). En el sistema implementamos CreadorLectores.cs, una factoría real que resuelve qué lector instanciar según la extensión del archivo (.json o .xml). Crear factorías cascarón para instanciar clases que los DTOs ya conocen violaría la regla rectora de la Sesión 15 (diapositiva 14): *'Trabajen sobre el código tal como está: no agreguen clases ni métodos que no existan'*.

B. ¿Por qué NO existe una clase intermedia 'AcuerdoComercial'?

La entidad Establecimiento mantiene una asociación directa con ITarifa (mediante _tarifaAcordada). Introducir AcuerdoComercial solo para contener la tarifa sin tener lógica propia constituiría el antipatrón *Middle Man* (indirección innecesaria). La relación directa respeta KISS y el patrón Strategy.

C. ¿Por qué 'Acontecimiento' es una clase simple sin jerarquía de subclases?

En POO, la herencia se justifica cuando existen **comportamientos diferenciados**. Un acontecimiento es un **registro inmutable de datos (Event Log)** con 4 campos idénticos (id, guía, fecha y código). No posee lógica propia. La máquina de estados y las transiciones pertenecen a Despacho.ProcesarAcontecimiento(). El polimorfismo se aplica donde aporta valor: en las tarifas (ITarifa) y en la evaluación de SLA (ModalidadServicio).

D. Inclusión del bloque de persistencia y serialización en el diagrama UML

En el archivo Draw.io se incorporaron ILectorArchivo<T>, CreadorLectores, IExportadorInforme, ExportadorJson y ExportadorXml para visibilizar el cumplimiento formal de serialización heterogénea y Factory Method exigidos en la rúbrica.

E. Regla de cátedra: Cero rombos en diagramas UML

Conforme a las diapositivas 4, 6 y 7 de la Sesión 15, el docente no utiliza rombos de agregación ni de composición. Toda relación estructural se modela exclusivamente mediante **líneas simples de asociación con multiplicidades explícitas** (ej. 1 a 0..*), herencia con flecha triangular cerrada y realización de interfaz con línea discontinua.

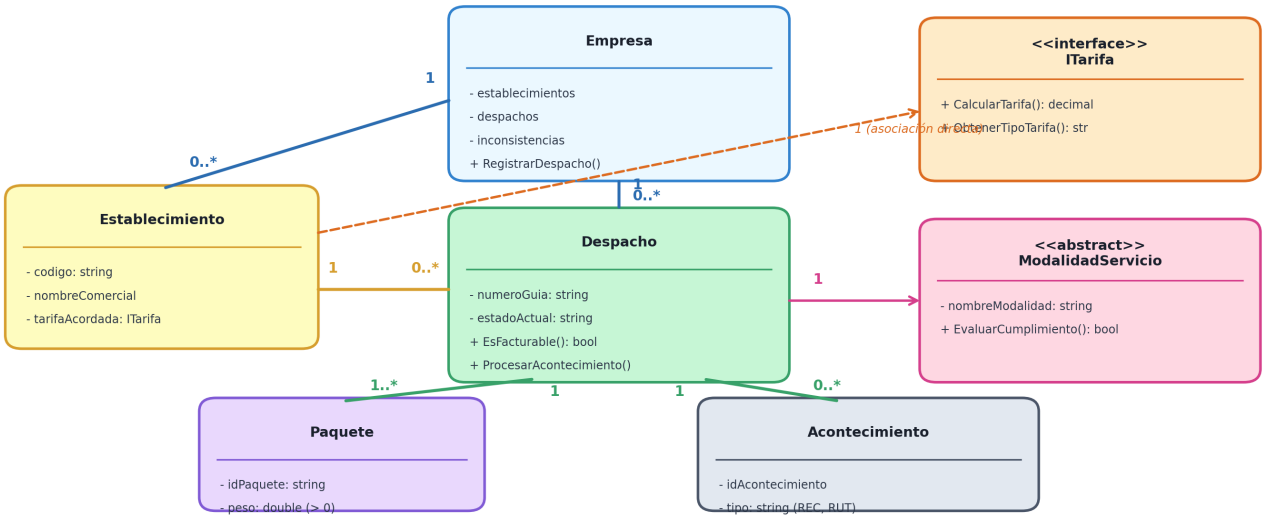


Figura 1: Modelo de Objetos de Dominio limpio, con asociaciones directas sin rombos (norma Sesión 15).

3. Anatomía de la Solución Limpia en C#

A. Desintoxicación de la clase 'Empresa' (Opción A)

La clase Empresa representa a la entidad raíz del negocio en memoria. Originalmente acumulaba 576 líneas al asumir lectura de disco, mapeo de DTOs y consolidación de reportes. En lugar de borrarla —lo que hubiera degradado el sistema a código procedimental en Program.cs— se desintoxicó:

• **Mappers:** La lógica de conversión externa se trasladó a los DTOs (MapToDomain()).
• **Informes:** La agregación de métricas se encapsuló en InformeOperativo.Crear(), InformeFinanciero.Crear() e InformeInconsistencias.Crear().

• **Resultado:** Empresa.cs quedó reducida a dominio puro, gestionando colecciones y coordinando seguimientos.

B. DTOs como puente desacoplado (Sesión 16 - Serialización)

Los DTOs (PedidoJsonDto, AcuerdoDto, etc.) reciben la estructura física del archivo. Mediante sus métodos MapToDomain(), construyen las entidades de negocio. Esto aísla a Despacho y Paquete de los formatos de serialización.

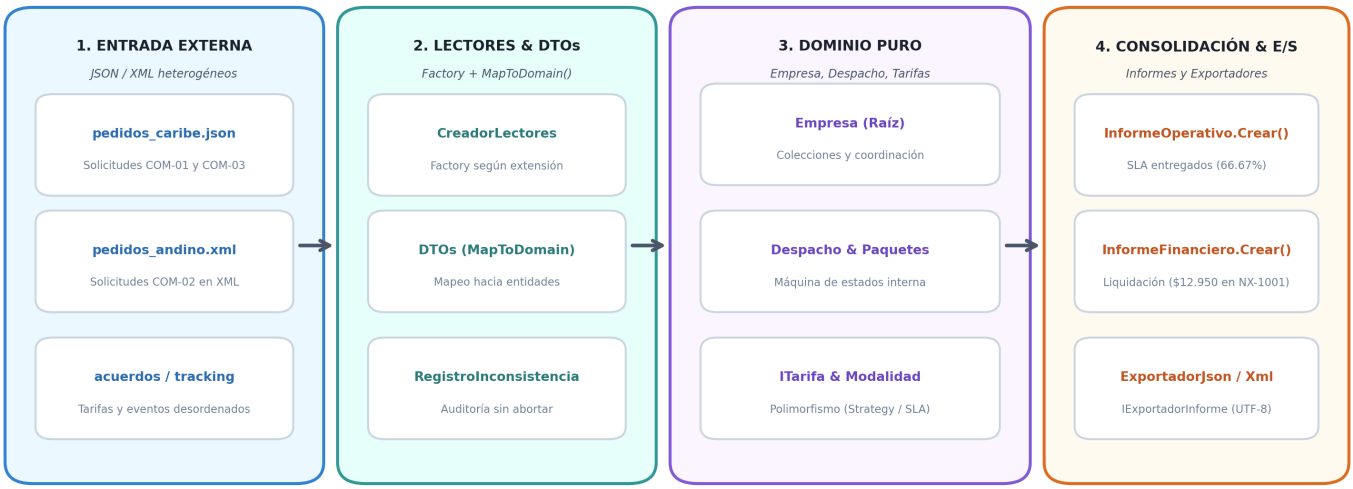


Figura 2: Arquitectura en 4 Capas de NexoLog. Flujo unidireccional de datos desacoplado.

C. Constructores privados sin parámetros (private MiClase() { })

■■ Fundamento Técnico del Constructor Privado

• **Soporte de Serialización y Reflexión:** Los deserializadores de .NET requieren un constructor sin parámetros para instanciar el objeto antes de poblar sus propiedades.
• **Protección del Encapsulamiento:** Al declararlo private, se impide que el código externo cree objetos en estado inválido (guías vacías, listas nulas). El código de negocio está obligado a usar el constructor público parametrizado que aplica validaciones estrictas (pesos > 0, cadenas no vacías).

D. Máquina de estados en Despacho.cs

Todo despacho nace en estado Pendiente en su constructor. Al recibir acontecimientos ordenados temporalmente, ejecuta transiciones protegidas:

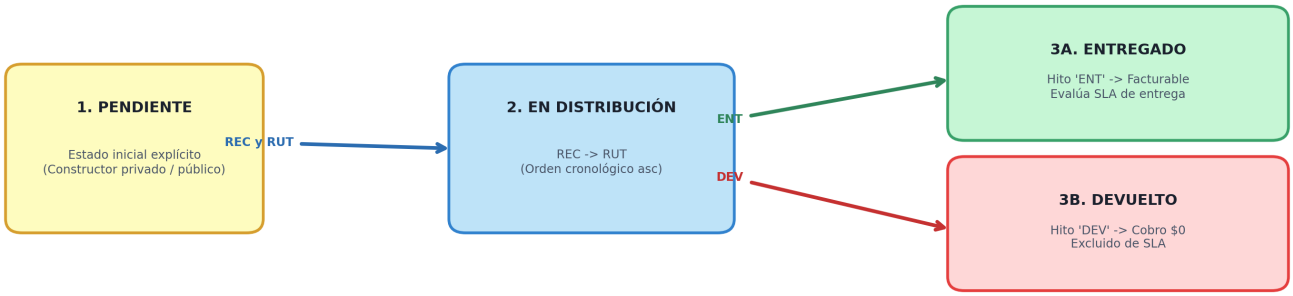


Figura 3: Ciclo de vida gobernado en Despacho. Solo el estado final Entregado habilita la liquidación de la tarifa.

4. Preguntas Clave del Sistema y Reglas de Cátedra

Aspecto del Sistema	Justificación Técnica
1. ¿Por qué ordenar cronológicamente los acontecimientos?	Los operadores logísticos generan archivos con eventos desordenados. Si se procesa una entrega antes de la salida a ruta, el despacho rechazaría el evento por incongruencia de secuencia. Ordenar por FechaHora ascendente reconstruye el orden causal real.
2. ¿Cómo funciona la resiliencia ante errores?	Las entidades de dominio lanzan NexoLogException si detectan datos inválidos. La capa de ingesta envuelve la operación en try/catch, registra la anomalía en RegistroInconsistencia y continúa procesando el resto del archivo sin interrumpir la ejecución.
3. ¿Por qué usar Strategy en ITarifa?	Cada comercio posee una política tarifaria distinta. Strategy encapsula la fórmula (por peso, distancia o fija) bajo una interfaz polimórfica, permitiendo incorporar nuevas tarifas sin modificar Despacho ni Establecimiento (Principio Abierto/Cerrado).
4. ¿Por qué usar Factory en CreadorLectores?	Centraliza la decisión de instanciación según la extensión (.json o .xml). La capa de aplicación desconoce los detalles de deserialización y solo interactúa con el contrato ILectorArchivo.
5. ¿Cómo se calcula el porcentaje de cumplimiento SLA?	El SLA se calcula exclusivamente sobre el universo de despachos entregados: (Entregados a Tiempo / Total Entregados) * 100. Los despachos pendientes o devueltos no forman parte del denominador.

Regla de Cátedra	Restricción	Implementación en NexoLog
Cero 'readonly'	Prohibido en la cátedra.	Toda protección se modela con campos privados y propiedades con solo 'get'.
Cero 'enum'	Constantes string.	Estados y tipos se definen como constantes 'public const string'.
Declaraciones individuales	Una línea por variable.	Prohibido agrupar variables ('double a, b;'). Cada variable en su línea independiente.
Cero 'switch'	Estructuras if/else.	Todas las decisiones condicionales se implementan con 'if / else if / else'.
Codificación UTF-8	Obligatoria en I/O.	Toda lectura o escritura de archivos especifica explícitamente Encoding.UTF8.
Cero rombos en UML	Convención de cátedra.	Relaciones modeladas con líneas directas de asociación y multiplicidades.

■ Cifras de Validación del Negocio (Golden Baseline)

- **Despacho Control NX-1001:** 3.7 kg × $3.500/kg (Comercio Caribe COM-01) = **$12.950** exactos.
- **SLA Global Entregados:** 4 a tiempo de 6 entregados = **66.67%**.
- **Total Facturado Consolidado: $104.450** sobre despachos con estado final Entregado.
- **Inconsistencias Auditadas:** 17 registros capturados sin caídas del sistema.